

Sreeraaghav Raja

2/13/26

40990068

EE Design 1 Technical Report

Microcontroller Basics

Introduction

This module was a thorough introduction into the embedded systems / firmware design flow process with a microcontroller. A microcontroller is a device with multiple modules that connect to a microprocessor (single-chip CPU) like the **RP2350**. The lab focused entirely on programming the **Raspberry Pi Pico 2 Microcontroller** and using it for two simple processes: blinking an LED and toggling a Piezo Buzzer.

Methods

Background Information:

This design implements the Raspberry Pi Pico 2 microcontroller alongside a LED, piezo buzzer, a potentiometer, a switch, and 2 resistors. The project is deliberately simple to flatten the curve of the embedded design process. Therefore, the number of components is much smaller than for later projects or even the final project for this course.

The most important component is the Raspberry Pi Pico 2 which is a microcontroller board that houses the **RP2350 microcontroller chip**. The RP2350 is a more powerful version of the RP2040 chip on the Raspberry Pi Pico that has these specifications (specs):

- Dual-Cortex M33 Processors at 150MHz
- 520 kB of on-chip SRAM
- GPIOs are 5 V-tolerant (powered) and 3.3 V-failsafe (unpowered)
- Peripherals:
 - 2 UART Channels
 - 2 SPI Channels
 - 2 I2C Channels

Ultimately, these specifications are not the most vital when programming the Pico 2 for this lab, but they are important as they provide a greater insight into the internal architecture of a complex device like the RP2350 [1].

Now, the term “microcontroller” has been used a few times up until this point without much of an explanation, so the following sentences will provide more information on that term. A microcontroller is very similar to a Central Processing Unit (CPU). The CPU is a part of a computer/system that is effectively “the ‘brain’ of a device” by performing arithmetic calculations, executing instructions, and controlling dataflow between the main device and peripherals [2]. The CPU is for a complete computing system and is a much larger scale than a smaller microcontroller. A

microcontroller is essentially a CPU on an IC chip that also controls the Input/Output (IO) interface [3]. This makes a microcontroller a fundamental asset with a much smaller Non-Recurring Engineering (NRE) cost as it can be efficiently reproduced to be applied to a variety of situations. For example, the Raspberry Pi Pico 2 W is often also used for wireless capabilities, but it has the same chip as the regular Pico 2. The only difference lies in the peripherals on each board since the Pico 2 W has a powerful Infineon CYW43439 Chip that allows for BLE (Bluetooth Low Energy), Bluetooth, and Wi-Fi capabilities [4]. Therefore, microcontrollers can be flexible and powerful enough that they can work for a variety of different peripherals if they allow **pin multiplexing**, being able to configure a pin's use for a different module. **These differences are emphasized by the different pinouts for Pico 2 and Pico 2 W as shown below.**

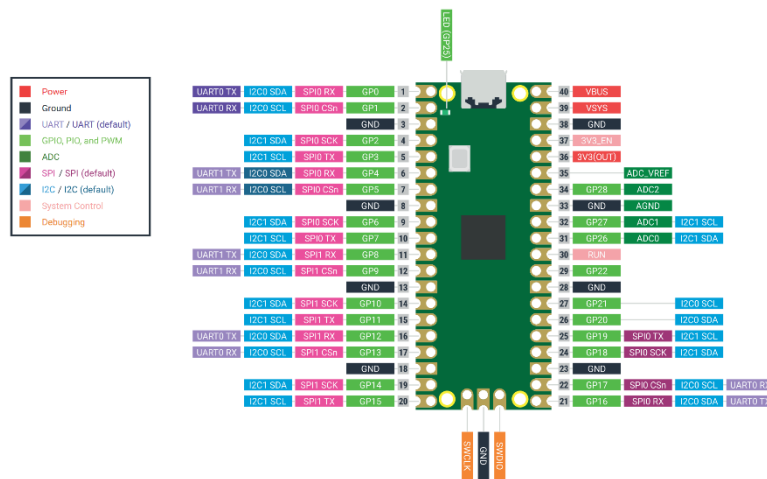


Figure 1: Raspberry Pi Pico 2 Pinout

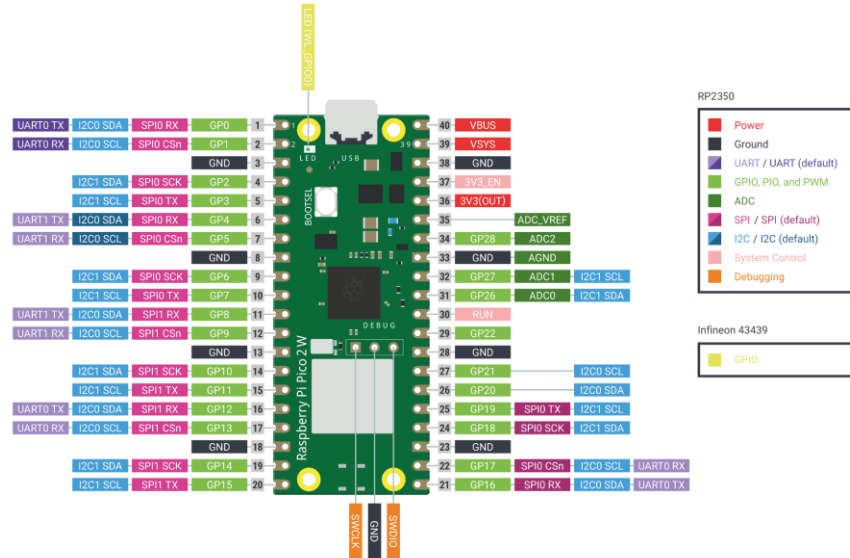


Figure 2: Raspberry Pi Pico 2 W Pinout

Apart from the microcontroller, the switch, piezo, and LED were the most important devices used for this lab. The LED is simple since it is a direct semiconductor that releases energy primarily as light which is useful for the case of the Red LED used in this lab. The switch is a device that can open/close circuit connections. The switch used was wired in active-high configuration as shown in the image below. An **active-high/pullup** switch configuration defaults to VCC/HIGH (the supply voltage) when the circuit is open, the switch is not pressed, and GND/LOW when the circuit is closed, the switch is pressed. This is advantageous because it can both provide a default state for a signal and save energy as power is only dissipated across a resistor when the switch is pressed as no current passes through the resistor in the default case. Additionally, switches are often made badly, so they bounce, oscillate in value before settling at a final value, so they require **debouncing** which can be done via several methods, but **hardware interrupts are the most common**. The last important component in this design is the piezo buzzer which implements the fundamental piezoelectric effect but in reverse. Instead of a device under mechanical pressure producing electrical charge, the presence of electrical charge deforms a mechanical device and causes it to produce sound [5]. This is a simple explanation of each of these devices, but their specs are not vital in understanding the design flow of this project.

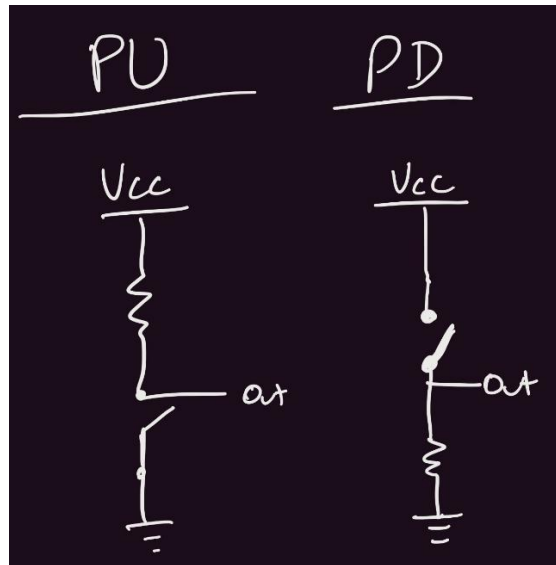


Figure 3: Pullup (PU) and Pulldown (PD) Switch Configurations

Debouncing is a significant part of this project that deals with the switch which is used in handling state transitions in the program. There are two ways to handle debouncing for a switch: **polling** and **hardware interrupts**. Polling is an easy way of handling debouncing as it constantly checks whether a switch is “pressed / released” for a set amount of time. However, this requires the programmer to know the time by which the switch will be released beforehand and the constant checking wastes CPU cycles. Hardware interrupts wait for a flag to trigger before stopping the program and performing a certain calculation / subprogram within an **interrupt handler** before resuming the main program again. Interrupts are more complex to implement than polling, but they save CPU cycles by letting the program continue its normal operation until an event triggers the flag. Moreover, interrupts remove any need for guesswork on the debounce time as they will wait for a flag to trigger the GPIO to trigger them.

Design Process Methodology:

The dogma behind the design process for this design was to incorporate all the background information to create a power-efficient and code-efficient design. For instance, the resistors used incorporated prior information about LEDs, Piezo Buzzers, and Switches. The resistor values were the only components that were based on trial and error. LEDs require a current-limiting resistor to prevent component degradation, so a 100Ω resistor was used to set the LED's brightness at a level that aligned with user preference. Moreover, the buzzer required a potentiometer to control its volume. The Pull-Up resistor value could be anything, but a greater resistance value would limit the current draw / power dissipated by the resistor. This was such a minor power draw that $2.2k\Omega$ was set as a suitable value for the resistor.

In terms of code efficiency, the Raspberry Pi Pico 2 supports MicroPython or C/C++. The Pico supports has its own C/C++ SDK since it is much more complex than MicroPython libraries. I opted to use C/C++ to program the Pico for my own learning experience and just to become more comfortable

working on my own projects without as much guidance as I had in my classes. Therefore, I used the following steps as a template to familiarize myself with the Pico and create my design:

Design Steps

1. Read about each of the components that are used in the design: LED, Piezo Buzzer, Switch, Potentiometer, and Pico 2.
2. Set up the C/C++ SDK in Windows Subsystem Linux (WSL) as it reduces the complexity of installing the Pico dependencies like Ninja.
3. Set up the VSCode Environment for the Pico SDK / create the Makefile which helps the programming process with autofill (checks for functions / defines that exist in the SDK)
4. Read examples of Pico code to understand the SDK and its functions + read the RP2350 documentation to understand the processor's limitations.
5. Design the full circuit modularly: first design the LED blinking, then the buzzer toggling, and finally the switch circuit since it also has the additional complexity of toggling between states.
6. Measure the frequencies and periods of each of the circuits to verify that they are within expected bounds of the design specifications: the **LED blinking at 2 Hz** and **Piezo Buzzer toggling at 2KHz**.
7. Design the **Altium Schematic and PCB Design** for future practice.

The Altium schematic was difficult to create because the correct pushbutton was difficult to find. The one in the Junior Design kit is the B3F-1000, so I had to search for this part with the *Manufacturer Part Search* function in Altium. The PCB was trivial to make since this design did not have many components which would add to the complexity of the design. Moreover, I had a difficult time understanding how to switch the footprint because I imported the schematic library file instead of the Intlib file. Otherwise, the rest of the design such as the creation of the hardware block just matched the schematic but was much less specific.

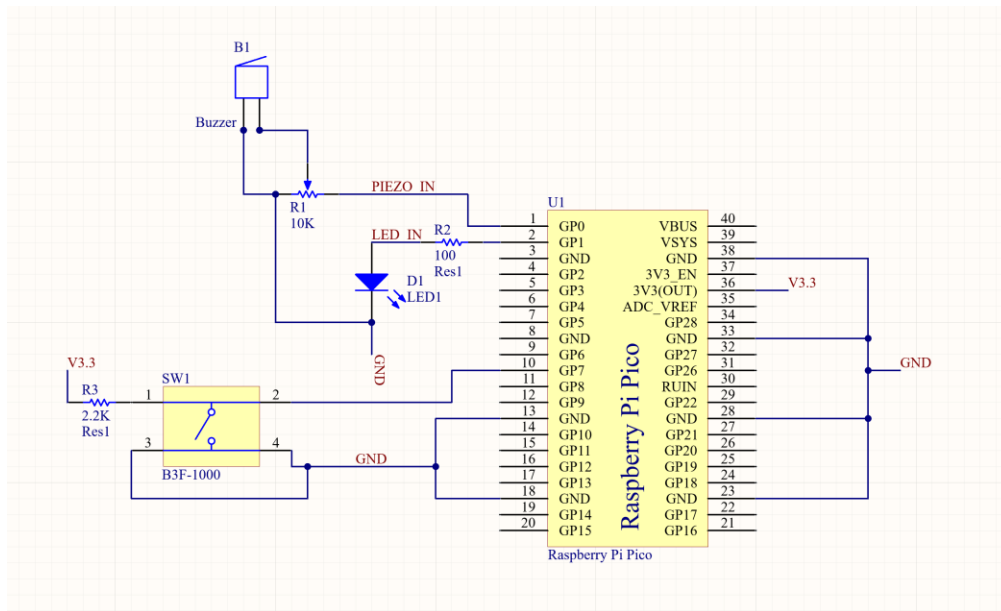


Figure 4: Altium Schematic

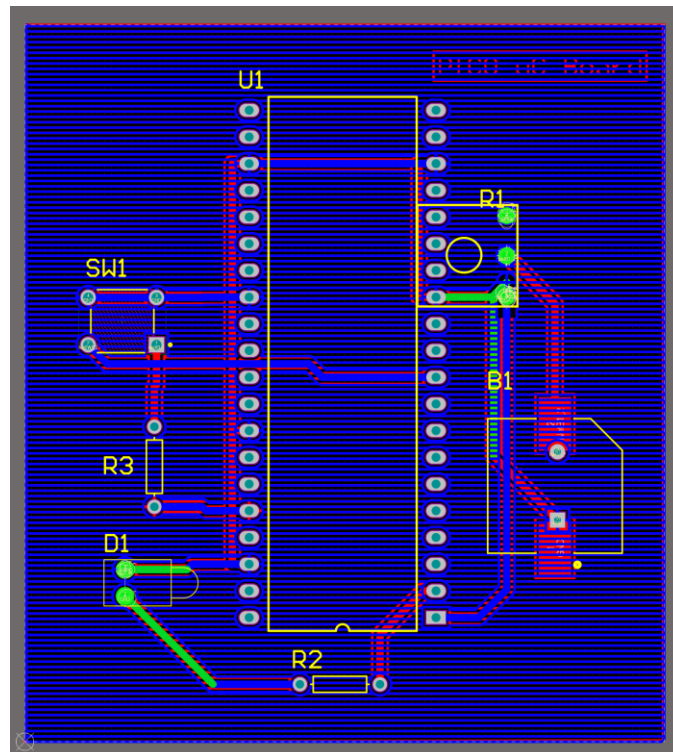


Figure 5: PCB Design

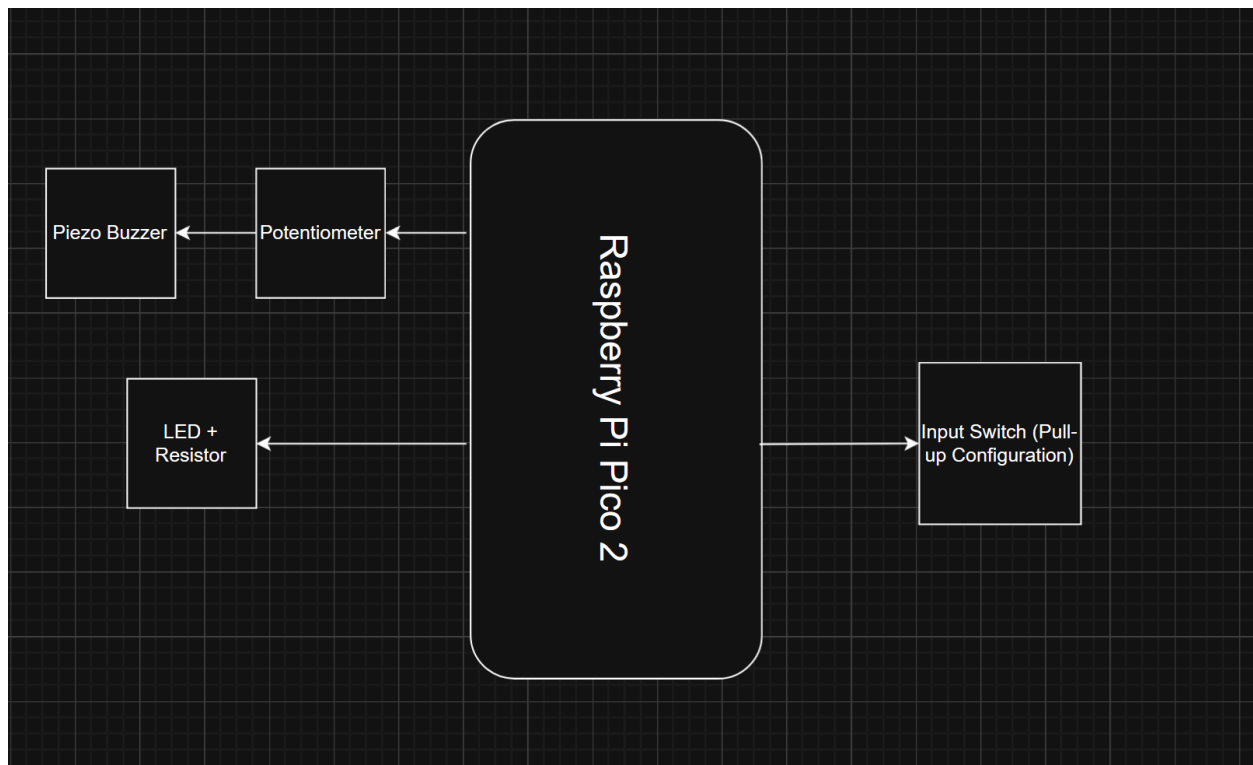


Figure 6: Hardware Block Diagram

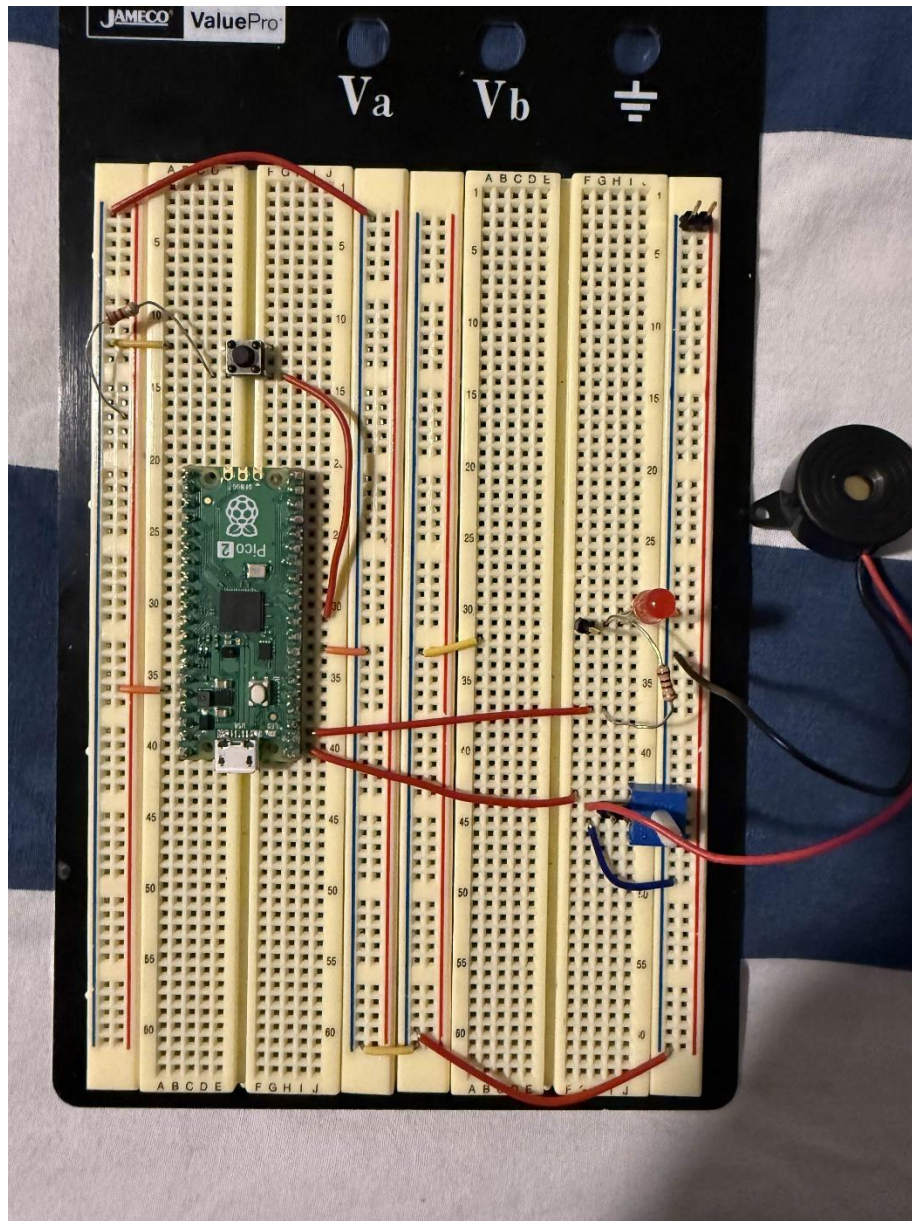


Figure 7: Breadboard Image

The code block was also relatively easy to make because of my prior experience with embedded programming. The block has 3 parallel processes: the main program, the program involving the interrupt, and the program involving the active timer. The active timer is part of the main program, but it does not affect the main program until that timer reaches its set timer (e.g. the counter value) when it resets. The alarm is different from a repeated timer because it can only run once before it must be reinitialized and it was my first time ever dealing with a construct like this before. Finally, the interrupt stops both the main program and the active timer when it is running as that is the functionality of an interrupt, so it is a single process. Therefore, the program oscillates between two parallel processes running simultaneously to one process that runs at a certain user-based time.

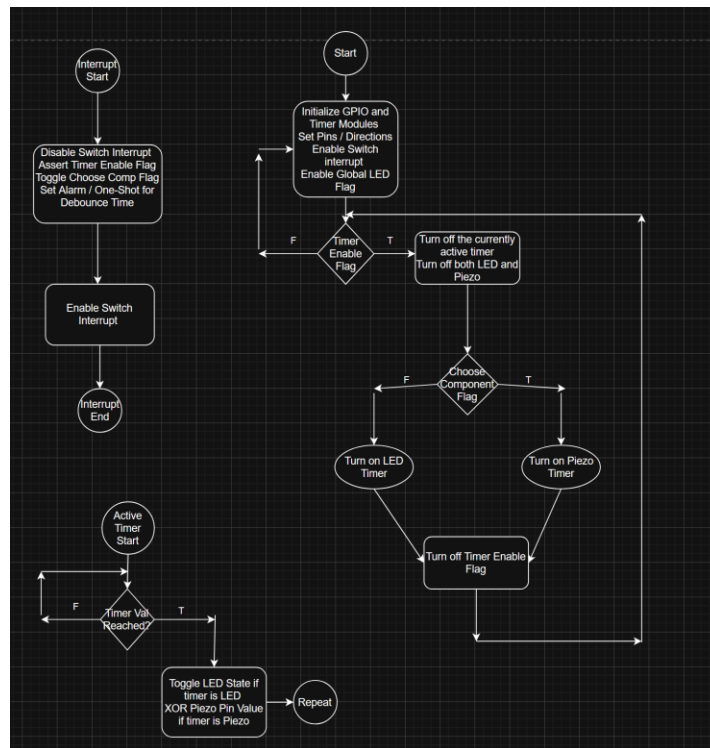


Figure 8: Code Block

Results

The results of the design matched the expected specifications of the project. The waves were all accurate too, but the piezo buzzer produced an interesting result. The piezo would produce a stable waveform when the resistance was turned all the way up on the potentiometer (saturated according to Figure 11). However, when unsaturated, the frequency would remain the same, but the duty cycle is off. Instead of being at 50%, the duty cycle is unreadable. This could be due to a variety of reasons, but it is most likely that the deformation of the internals of the piezo causes this noise. Moreover, the frequencies are all defined by the repeating timer constructs but must be twice as fast as specified to account for the LED being both on and off. So, the timer must be set to have a frequency of 4KHz if the specified piezo frequency is 2KHz. For more information on interrupts, please read the methodology section as this will account for the why the timer is accurate for my design.

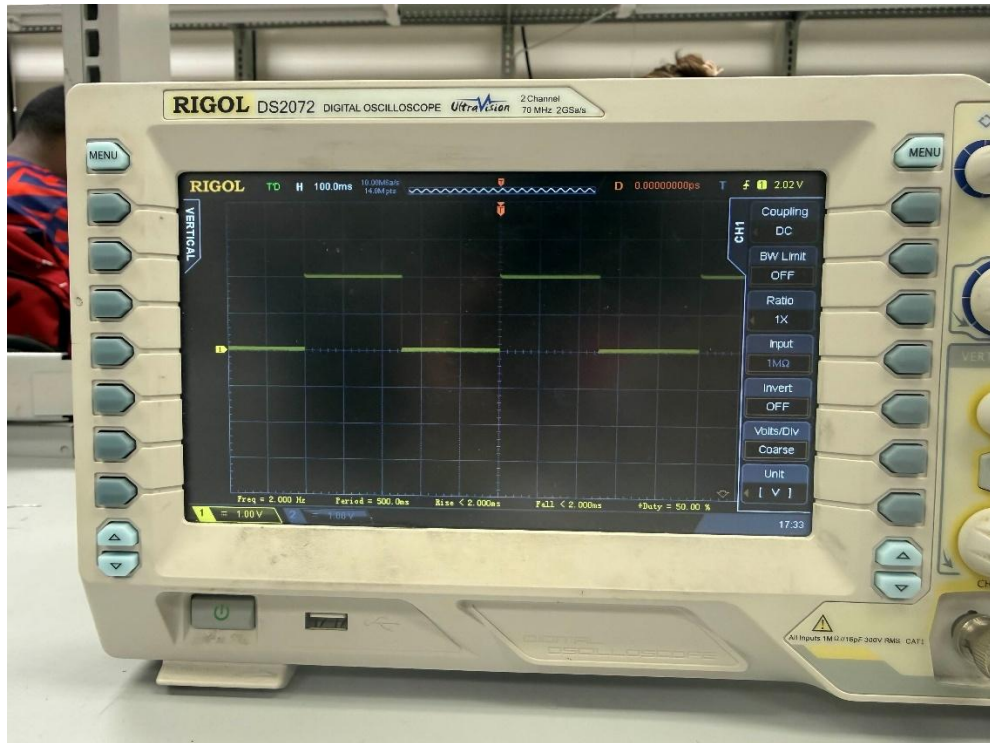


Figure 9: LED Waveform Output

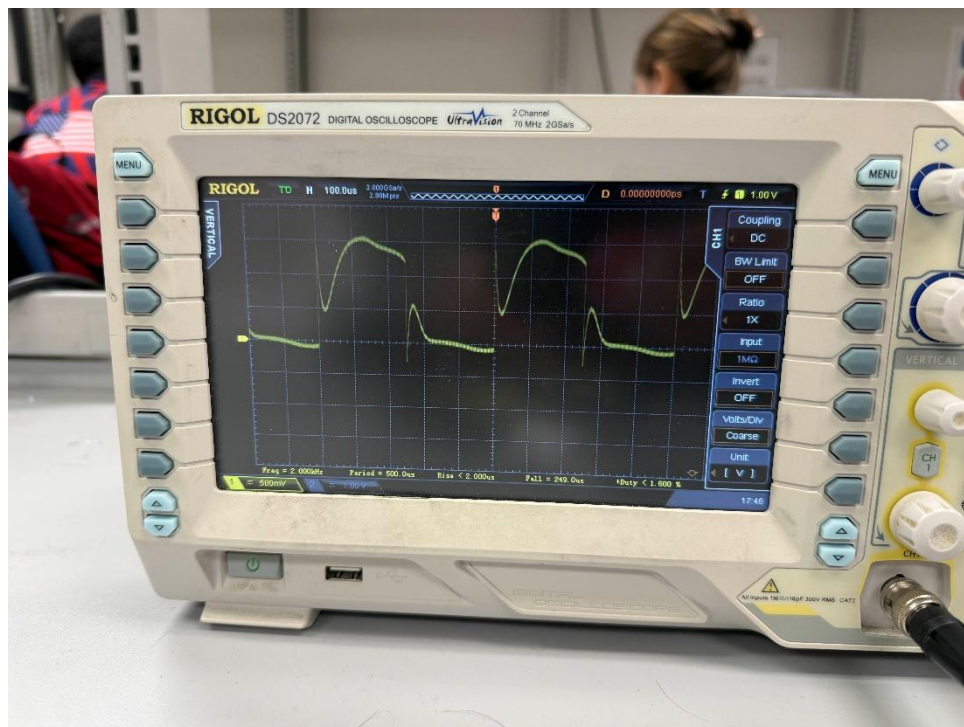


Figure 10: Piezo Unsaturated Waveform Output

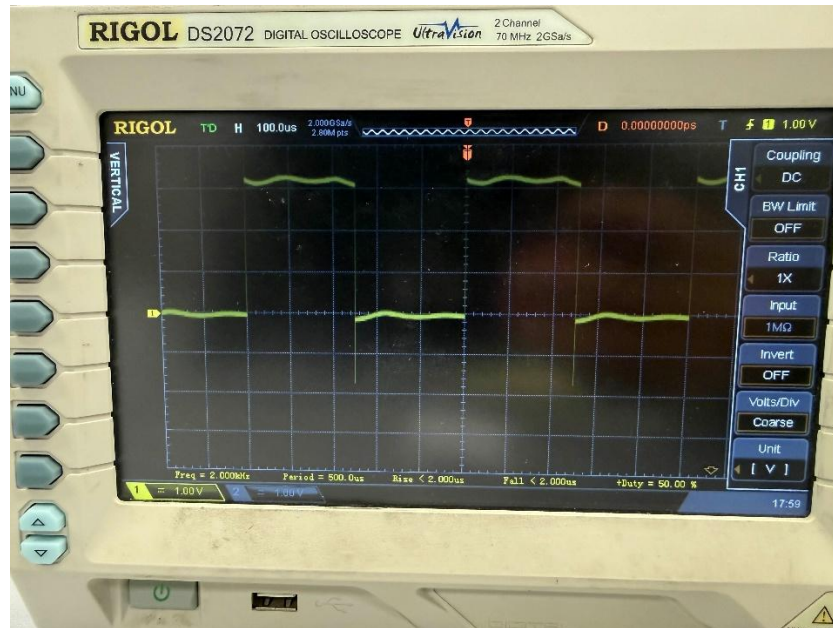


Figure 11: Piezo Saturated Waveform Output

Conclusion

This lab focuses on microcontrollers which I already have experience with, but this was more interesting as I learned new constructs for the Raspberry Pi Pico 2. I also learned how to interface with a Pico using Ubuntu which is more useful for industry.

References

- [1] "RP2350 Datasheet: A microcontroller by Raspberry Pi," Raspberry Pi, Aug. 2024. Accessed: Feb. 15, 2026. [Online]. Available: <https://pip-assets.raspberrypi.com/categories/1214-rp2350/documents/RP-008373-DS-2-rp2350-datasheet.pdf?disposition=inline>
- [2] Arm Ltd, "What Is a Central Processing Unit? – Arm®," *Arm | the Architecture for the Digital World*. <https://www.arm.com/glossary/cpu> (accessed Feb. 15, 2026).
- [3] GeeksforGeeks, "Microcontroller and its Types," *GeeksforGeeks*, Feb. 20, 2022. <https://www.geeksforgeeks.org/digital-logic/microcontroller-and-its-types/> (accessed Feb. 15, 2026).
- [4] Raspberry Pi, "Pico-series microcontrollers - Raspberry Pi documentation," *Raspberrypi.com*, 2024. <https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html> (accessed Feb. 15, 2026).

[5] P. Kendall, "How Do Piezo Buzzers Work | RS," *uk.rs-online.com*, Oct. 30, 2023. <https://uk.rs-online.com/web/content/discovery/ideas-and-advice/how-do-piezo-buzzers-work> (accessed Feb. 15, 2026).

[6] J. Hopkins, "Polling vs Interrupts: Exploring their Differences and Applications," *Total Phase Blog*, Oct. 18, 2023. <https://www.totalphase.com/blog/2023/10/polling-interrupts-exploring-differences-applications/> (accessed Feb. 15, 2026).

Appendix

```
#include <stdio.h>
#include <stdbool.h>
#include "pico/stdlib.h"
#include "hardware/uart.h"
#include "hardware/timer.h"
#include "pico/time.h"
#include "hardware/gpio.h"
#include "hardware/irq.h"

#define DIODE_PIN 1
#define PIEZO_PIN 0
#define SWITCH_PIN 7

#define HZ_2_IN_MS -250 // timer has to toggle every 250 ms
#define KHZ_2_IN_MS -250 // timer has to toggle every 250 us
#define DEBOUNCE_TIME 100 // typical debounce time is 10 ms
// #define LED_PIN 25>
// #define UART_TX_PIN 0
// #define UART_RX_PIN 1
// #define BAUDRATE 115200

volatile bool led_state = false, timer_enable = true, choose_comp = false;

bool toggle_diode(__unused struct repeating_timer *t){
    led_state ^= 1;
    gpio_put(DIODE_PIN, led_state);
    return true;
}

bool toggle_piezo(__unused struct repeating_timer *t){
    gpio_xor_mask64(1u << PIEZO_PIN);
    return true;
}
```

```

int64_t gpio_alarm_callback(alarm_id_t id, __unused void *user_data){
    // re-enable the IRQ after the alarm goes off
    gpio_set_irq_enabled(SWITCH_PIN, GPIO_IRQ_EDGE_FALL, true);
    return 0;
}

void gpio_irq_callback(uint gpio, uint32_t event_mask){
    // disable the interrupt
    gpio_set_irq_enabled(SWITCH_PIN, GPIO_IRQ_EDGE_FALL, false);

    // make it possible to change timers
    timer_enable = true;

    // choose the timer and make it different from the previous one
    choose_comp ^= 1;

    // ignore the callback when the alarm isn't set (should test with true too)
    add_alarm_in_ms(DEBOUNCE_TIME, gpio_alarm_callback, NULL, false);
}

int main(void){
    stdio_init_all();

    // initialize all three pins
    gpio_init(DIODE_PIN);
    gpio_init(PIEZO_PIN);
    gpio_init(SWITCH_PIN);

    // set pin directions
    gpio_set_dir(DIODE_PIN, GPIO_OUT);
    gpio_set_dir(PIEZO_PIN, GPIO_OUT);
    gpio_set_dir(SWITCH_PIN, GPIO_IN);

    // can internally define this pin as a pull-up switch config
    gpio_pull_up(SWITCH_PIN);

    // initialize the struct for the diode and piezo timers
    struct repeating_timer active_timer;

    // add a repeating timer with callback for the diode and piezo
    add_repeating_timer_ms(-250, toggle_diode, NULL, &active_timer);

    // enable the falling edge interrupt for the switch pin (enable the
    interrupt)

```

```

gpio_set_irq_enabled(SWITCH_PIN, GPIO_IRQ_EDGE_FALL, true);
gpio_set_irq_callback(gpio_irq_callback);
irq_set_enabled(IO_IRQ_BANK0, true);

for(;;){
    if(timer_enable){

        cancel_repeating_timer(&active_timer);

        // turn off both components (for a very small unit of time)
        gpio_put(PIEZO_PIN, 0);
        gpio_put(DIODE_PIN, 0);

        if(!choose_comp){
            // add the diode timer
            add_repeating_timer_ms(-250, toggle_diode, NULL, &active_timer);
        }
        else{
            // add piezo timer
            add_repeating_timer_us(-250, toggle_piezo, NULL, &active_timer);
        }

        timer_enable = false;
    }
}

return 0;
}

```